Data Driven Reachability Analysis toolbox using Python

by

Bouhelal Hamza

Bachelor Thesis in Computer Science

Submission: August 27, 2022 Supervisor: Prof. Dr. Amr Alanwar

Jacobs University Bremen | Department of Computer Science and Electrical Engineering

Statutory Declaration

Family Name, Given/First Name	Hamza, Bouhelal
Matriculation number	30002644
Kind of thesis submitted	Bachelor Thesis

English: Declaration of Authorship

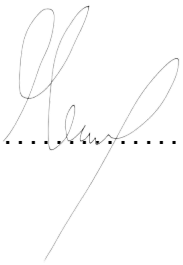
I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: ErklÃ¤rung der Autorenschaft (Urheberschaft)

Ich erklÃ¤re hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschlieÃŸlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprÃ¼ft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukÃ¼nftigen Arbeiten verglichen werden zu kÃ¶nnen. Dies berechtigt jedoch nicht zur Verwendung oder VervielfÃ¤ltigung.

Diese Arbeit wurde noch keiner anderen PrÃ¼fungsbehÃ¶rde vorgelegt noch wurde sie bisher verÃ¶ffentlicht.



27/08/2022
Date, Signature

Abstract

Reachability analysis is a mathematical technique used to determine whether a given point in a system can be reached from another given point. This is often used in the field of control theory to determine whether or not a given control system is stable. In this paper, we develop a Data-Driven reachability analysis toolbox that relies on data sampled from simulations of the system to perform the reachability analysis. More especially, The toolbox proposes implementation of algorithms that uses a set of sampled data to over-approximate the reachable set of systems. We demonstrate the tool's feasibility by applying it to several example systems and comparing the over-approximations computed using the above mentioned algorithms with the actual base model of the systems. Thus, providing theoretical guarantees for safety verification.

Contents

1 Introduction

Reachability analysis is a formal verification technique for determining whether a system can or cannot reach a certain state. It is often used to verify the safety of systems, such as checking that a system, starting from an initial state, will never reach an unsafe state under certain parameters and input. While most traditional approaches for reachability analysis rely on the knowledge of the dynamics of the system, Data Driven reachability analysis utilize data gathered from simulations of a system to compute the reachable set, making it possible for system that can be simulated to be over-approximated.

Researchers put out a number of formulations [1] in recent years that formalize reachability analysis's ability to guarantee an autonomous system's safety (i.e., for autonomous vehicles and drones). A reachable set of states is often calculated based on a model of the subject system using either set-propagation techniques or simulation-based techniques. The majority of methods compute over-approximations of the robot's reachable states in order to produce a reachable set that can be utilized to assure safety. These conventional methods, however, are susceptible to model error and do not take into account the easily accessible trajectory data that robots produce.

Dynamical systems are systems in which a function describes the time dependence of a point in an ambient space. dynamical systems always have a state that corresponds to a certain location in the suitable state space. A tuple of real numbers or a vector in a geometric manifold are frequently used to describe this state. The dynamical system's evolution rule is a function that specifies the outcomes of the current state on possible future states. The function is frequently deterministic, meaning that for a given time interval, only one possible future state can be deduced from the present condition [2] [3]. Some systems, however, exhibit stochastic behavior, in which the evolution of the state variables is also influenced by chance events.

In this paper, we propose the implementation (in Python) of algorithms [4] to over-approximate the reachable set of each of the following dynamical systems:

- Linear Time Invariant systems
- Lipschitz Nonlinear systems
- Polynomial systems

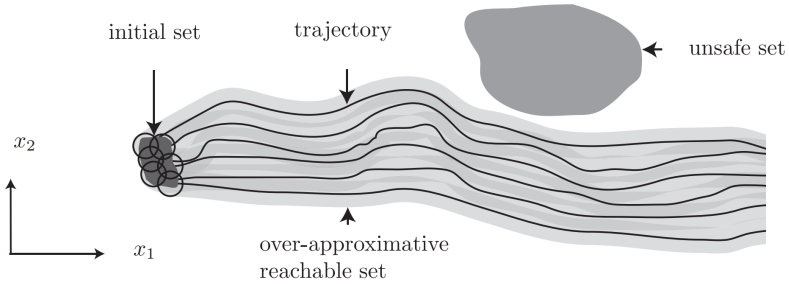


Figure 1: We aim to compute an over-approximation of the reachable set using data gathered from simulations of the system we are analysing. [5]

To perform this analysis, we will represent the sets using Zonotopes [6], taking advantage

of the properties of this set representation. Included in the toolbox are utilities needed to complement the set representation implementation used, the implementation of 3 classes each providing methods to gather the data from simulation of the system and to compute an over-approximation of the reachable set of the analysed system, this, for each of the above mentioned dynamical systems.

2 Survey

2.1 Dynamical systems

Dynamical systems are defined as [7] those whose state changes with time (t). The study of dynamical systems is the focus of dynamical systems theory, which has applications to a wide variety of fields such as mathematics, physics,biology,chemistry, engineering, economics,history, and medicine. Dynamical systems are a fundamental part of chaos theory, logistic map dynamics, bifurcation theory, the self-assembly and self-organization processes, and the edge of chaos concept.

We must distinguish between two types of dynamical systems [7]:

- Continuous time variable t ($t \in \mathbb{R}$) where :

$$\frac{dx}{dt} = \dot{x} = X(x) \tag{1}$$

- Discrete time variable t ($t \in \mathbb{N}$ or $\mathbb{Z}$) where :

$$X_{t+1} = f(x_t) \tag{2}$$

Simulation of a system is the imitation of the operation of a real-world system over time we should compare simulation results against experimental results from the real system. For simulation purposes we are mainly interested in mathematical models, as opposed to physical or other simulation media.

2.2 Reachability Analysis Problem

We wish to compute and analysis the set of reachable states of a system, in order to decide if a given set of unsafe states is reachable starting from a set of initial conditions. This is known as the reachability analysis problem. Computing approximations of reachable sets can be useful for many purposes: [8]

- Formal verification: Given a system to be verified and a specification of liveness or fairness, one can generate a monitor automaton. This automaton can then be used to check whether the system satisfies some specification, the verification problem can be translated into a reachability problem.
- Computation of invariant sets and regions of attractions: as per [9], reachability analysis performs better compared to using Lyapunov functions at computing invariant sets (sets useful to ensure stability of model predictive control).
- Robust control: Reachability analysis has had a big impact on control theory, it ensures that controllers reach their goal state while avoiding the unsafe state, this by restricting the input and taking into account the process noise.

- Set-based prediction: Prediction algorithms are often used by autonomous systems to avoid conflicts with surrounding entities. We can compute all possible reachable sets of each entity to ensure that no intersections occur at a certain time.

2.2.1 Definition

Given a dynamical system, an initial set X_0, and unsafe state set X_u and time horizon T, is there any trajectory starting from X_0 that intersets X_u within the time frame T ?

Reachability analysis can be done within either a finite or infinite time horizon, computing a finite number of reachable sets ahead of the initial set might seem restrictive, but proves itself to be sufficient:

- it is known that failures would manifest within a finite time horizon if at all
- in many cases the reachability analysis problem has uncertain time varying parameters that makes the model invalid for infinite time horizons
- the infinite time horizon problem is often harder to solve than the finite time horizon problem

2.2.2 Existing approaches

- Abstraction-Based Techniques: The goal of abstraction-based verification is to construct a finite-state abstraction of a system that can be refined, possibly using counterexamples. Once the abstraction is constructed, the reachability problem is solved on this abstraction. If the unsafe set in the abstract state-space cannot be reached, then it is concluded that the same is true for the original system. However, abstract counterexamples can be spurious and may not correspond to a real execution of the concrete system. This can be addressed by refining the abstraction to rule out such counterexamples.
- Dynamic Programming (Hamilton-Jacobi) Approaches: The dynamic programming approach to controlling hybrid systems is quite powerful, but solving the resulting partial differential equation can be expensive. It leads to a partial differential equation (PDE) that needs to be solved in order to compute a controllable region, which exclude the undesired states. It applies to nonlinear systems and can compute a set of control strategies for guaranteeing safety. However, solving PDEs requires expensive numerical methods whose complexity can be exponential in the number of state variables.
- Set-Propagation: Set propagation is a technique used to approximate the reachable sets of a system over a finite time horizon. The computed reachable set is represented as unions of sets in a chosen family (e.g., ellipsoids, polyhedra, Taylor models). Set propagation algorithms propagate these sets for a small time step Δ so that an approximation that is valid for time up to t is now valid for time up to $t+\Delta$, by repeatedly iterating this process, an over-approximation of the reachable sets up to a finite time horizon T is produced. Set propagation techniques have been investigated extensively for linear systems, and are currently capable of analyzing linear dynamical systems with more than a billion state variables, linear hybrid systems with hundreds of state-variables, and nonlinear systems with tens of state variables.

- Constraint Solving Approaches:There are a variety of approaches that can be used to show that no counterexample trace with a given length/time bound exists for a reachability problem. One important class of approaches uses constraint solvers to construct an abstraction. Another approach, called bounded-model checking, encodes the reachability problem as a set of constraints. More recently, Kong et al. have developed a tool called dReach which uses other reachability analysis tools for nonlinear dynamical systems to approximate the solution to ODEs.
- Falsification: falsification focuses on disproving correctness by searching for a counterexample that establishes that an unsafe state is reachable starting from some initial state. Recently, there have been many approaches towards falsification based on using robustness of trajectory (its minimum distance to the unsafe set) as a fitness function that is minimized repeatedly using optimization. Although they do not have guarantees of exhaustiveness, falsification techniques have been more successful in the industry wherein they provide a form of “smart fuzz-testing” for CPS. Ultimately, both verification and falsification are important in ensuring the safety and correctness of CPS systems.

3 Background

We first need to define some of the set representations needed to perform the analysis, then have a quick statement defining the problem statement and notations needed.

3.1 Zonotope

Formally, a zonotope [6] is a Minkowski sum of a finite set of line segments, in other words, it is a mathematical shape formed by adding together a finite number of line segments that all pass through a common point. Given a center $c \in \mathbb{R}^n$ (an n dimensional point) and a generator matrix $G = [g_1, \ldots, g_m]$ with g_i a generator vector and m the number of generators, we define a zonotope by $Z =< c, G >$ and further define:

$$Z = \{x \in \mathbb{R}^n | x = c + \sum_{i=1}^{m} \beta_i \times g_i, -1 \leq \beta_i \leq 1\} \tag{3}$$

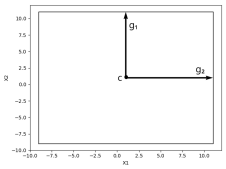
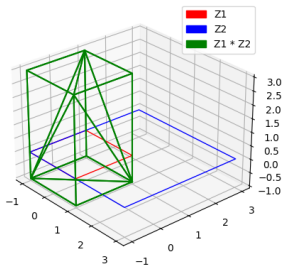
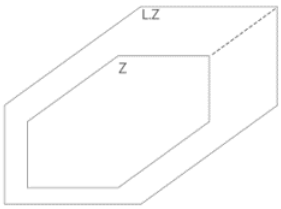


Figure 2: A two dimensional zonotope Z, with center c and generator vectors g_1 and g_2

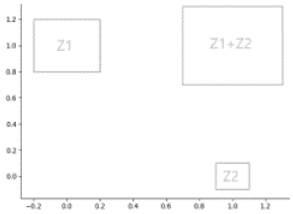
We define some of the properties of zonotopes we will make use of for our analysis:



Zonotope are closed under Cartesian Product: $Z_1 \times Z_2 =< \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}, \begin{bmatrix} G_1 & 0 \\ 0 & G_2 \end{bmatrix} >$



Zonotopes are closed under linear map: $LZ =< Lc, LG >$



Zonotopes are closed under Minkowski sum: $Z_1 \oplus Z_2 =< c_1 + c_2, [G_1, G_2] >$, We also denote: $Z_1 - Z_2 = Z_1 \oplus -1Z_2$

Furthermore, we introduce the concept of Zonotope reduction [10], in comparison to parallelotopes and ellipsoids, zonotopes are substantially more flexible while needing far less computation time than ordinary convex polytopes. However, a major drawback, especially for recursive algorithms, is that many operations on zonotopes produce outcomes that are far more complex than their inputs. Order reduction techniques were developed to solve this problem by enclosing a given zonotope within a less complex one. Many control algorithms depend on these techniques, which have a substantial impact on their effectiveness and performance. For instance, improper reduction can output an inaccurate, conservative result. We will make use of the Girard method [11] to perform the Zonotope reduction:

Given a zonotope $Z =< c, \begin{bmatrix} g_1 & ... & g_m \end{bmatrix} >$ and $n = dim_Z \times reduction\_order \leq m$, we will be picking the n generators with the smallest value of the score: $||g_i||_1 - ||g_i||_\infty$ to construct a reducted Zonotope :

$$Z_{red} =< c, \begin{bmatrix} g'_1 & ... & g'_n \end{bmatrix} > \qquad g'_i \in \{g_j \mid g_j \text{ included in the } n \text{ picked generators}\}$$
$$\forall i \in [1, ..., n]$$

In addition, we describe the method used to represent a Zonotope of dimension dim_Z as a group of 2 dimensional graphs, $\forall i \in [1, ..., floor(dim_Z/2) + (1 \text{ if } dim_Z \ \% \ 2 = 1)]$ each graph i representing the Zonotope at dimensions $2 \times i$ and $2 \times i + 1$.

We first need to compute all the extremities of the zonotope, to do this we need to consider all the combination of the sum of all generators scaled with β_i = 1 or -1, this, shifted by c, the center of the zonotope. Then, compute the convex hull of those points to get the final outline needed to visualize the Zonotope.

Algorithm 1 Plot Zonotope 2D

Input: The Zonotope Z.

$Gens \leftarrow Z.generators()$
$Nbre\_of\_gens \leftarrow Gens.length$
$even \leftarrow (floor(Z.dim/2) == Z.dim/2) \ ? \ true : false$
for $i \leftarrow 0$ to $(even \ ? \ floor(Z.dim/2) : floor(Z.dim/2) + 1)$ **do**
  $result \leftarrow [\,]$
  $Ti \leftarrow (\ !even$ and $i == floor(Z.dim/2) + 1) \ ? \ i - 1 : i$ ▷ Defines dimension to plot
  for $j \leftarrow 0$ to $2^{Nbre\_of\_gens}$ **do**
    $Cur\_Point \leftarrow Z.center()$
    $Combs \leftarrow j.toBinary().fill(Nbre\_of\_gens)$
    for $k \leftarrow 0$ to $Nbre\_of\_gens$ **do**
      $Cur\_Point \leftarrow Cur\_Point + (Combs[k] == 1 \ ? \ 1 : -1) \times Gens[2Ti : 2Ti+2][k]$
    end for
    ▷ Compute current point $\quad combs = 0011 \Rightarrow curr\_point = c - g_0 - g_1 + g_2 + g_3$
    $result.append(Cur\_Point)$
  end for
  $extremities \leftarrow Convex\_Hull(result)$
  $extremities.append(extremities[0])$ ▷ Creates close loop
  $plot(extremities)$
end for

We can notice the high complexity of **Algorithm 1**, making apparent how crucial Zonotope reduction techniques are.

3.2 Matrix Zonotope

Matrix hypercubes [5] are composed of real-valued matrices $\tilde{A}^{(i)}$, together, a parameter vector β and a set of matrices $\hat{A}^{(i)}$ define the system matrix $A(\beta)$:

$$A(\beta) = \sum_{i=1}^{k} \beta_i \times \hat{A}^{(i)}, \qquad \beta_i \in [\underline{\beta}, \overline{\beta}]$$

We consider the normalized version of $A(\beta)$: $\beta_i \in [-1, 1]$.

Due to the similarity of the mathematical definition of Zonotopes and the the normalized version of a matrix hypercube, we will use the notation Matrix Zonotope, we denote:

Given a center $C_M \in \mathbb{R}^{n \times T}$ and $\gamma_M \in \mathbb{N}$ generator matrices $\tilde{G}_M = [G_M^1, ..., G_M^{\gamma_M}] \in \mathbb{R}^{n \times (T \times \gamma_M)}$, we define a matrix zonotope [4] by $M =< C_M, \tilde{G}_M >$ and further define:

$$M = \{X \in \mathbb{R}^{n \times T} | X = C_M + \sum_{i=1}^{\gamma_M} \beta_i \times G_M^i, -1 \leq \beta_i \leq 1\} \tag{4}$$

3.3 Interval Matrix

We will define Interval Matrices as per [4]:

An interval matrix I specifies the interval of all possible values for each matrix element between the left limit $\underline{I}$ and right limit $\overline{I}$:

$$I = [\underline{I}, \overline{I}], \qquad \underline{I}, \overline{I} \in \mathbb{R}^{n \times n}$$

We denote the conversion of matrix zonotope M to an interval matrix I by: $I = IntervalMatrix(M)$ and vice versa: $M = Zonotope(I)$.

3.4 Data-Driven Reachability Analysis

We consider the discrete time system:

$$x(i+1) = f(x(i), u(i)) + w(i) \qquad y(i) = x(i) + n(i) \tag{5}$$

With $u(i)$ and $w(i)$ representing respectively the input and the noise applied to the system at step i, given that $u(i)$ bounded by an input Zonotope Z_u and $w(i)$ bounded by a noise Zonotope Z_w. $y(i)$ the measured state corrupted by $n(i)$, the measurement noise. $x(0) \in X_0$ the initial state of the system, bounded by the initial set X_0.

We consider K input-state trajectories each of length T gathered from simulations of a system of dimension Dim_X and of input dimension Dim_U, we re-organize this data like so:

$$U_- = \begin{bmatrix} u_1^{(1)}(0) & ... & u_1^{(1)}(T-1) & u_1^{(2)}(0) & ... & u_1^{(2)}(T-1) & ... & u_1^{(k)}(0) & ... & u_1^{(k)}(T-1) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_{Dim_U}^{(1)}(0) & ... & u_{Dim_U}^{(1)}(T-1) & u_{Dim_U}^{(2)}(0) & ... & u_{Dim_U}^{(2)}(T-1) & ... & u_{Dim_U}^{(k)}(0) & ... & u_{Dim_U}^{(k)}(T-1) \end{bmatrix}$$

$$X = \begin{bmatrix} x_1^{(1)}(0) & ... & x_1^{(1)}(T) & x_1^{(2)}(0) & ... & x_1^{(2)}(T) & ... & x_1^{(k)}(0) & ... & x_1^{(k)}(T) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{Dim_X}^{(1)}(0) & ... & x_{Dim_X}^{(1)}(T) & x_{Dim_X}^{(2)}(0) & ... & x_{Dim_X}^{(2)}(T) & ... & x_{Dim_X}^{(k)}(0) & ... & x_{Dim_X}^{(k)}(T) \end{bmatrix}$$

With $u_{Dim}^{(i)}(j)$ representing dimension Dim of the input applied to the system for trajectory i at steps j, we also use the notation: $\{u_{Dim}^{(i)}(j)\}_{j=0}^{T-1}, i \in [1, ..., K], Dim \in [1, ..., Dim_U]$. $x_{Dim}^{(i)}(j)$ representing the state of the system at dimension Dim for trajectory i after j steps, we also use the notation: $\{x_{Dim}^{(i)}(j)\}_{j=0}^{T}, i \in [1, ..., K], Dim \in [1, ..., Dim_X]$.

We further define:

$$X_+ = \begin{bmatrix} x_1^{(1)}(1) & ... & x_1^{(1)}(T) & ... & x_1^{(k)}(1) & ... & x_1^{(k)}(T) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{Dim_X}^{(1)}(1) & ... & x_{Dim_X}^{(1)}(T) & ... & x_{Dim_X}^{(k)}(1) & ... & x_{Dim_X}^{(k)}(T) \end{bmatrix}$$

$$X_- = \begin{bmatrix} x_1^{(1)}(0) & ... & x_1^{(1)}(T-1) & ... & x_1^{(k)}(0) & ... & x_1^{(k)}(T-1) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{Dim_X}^{(1)}(0) & ... & x_{Dim_X}^{(1)}(T-1) & ... & x_{Dim_X}^{(k)}(0) & ... & x_{Dim_X}^{(k)}(T-1) \end{bmatrix}$$

We additionally denote the set $\hat{W}_-$ storing the process noise of the simulation by:

$$\hat{W}_- = \begin{bmatrix} w_1^{(1)}(0) & ... & w_1^{(1)}(T-1) & ... & w_1^{(k)}(0) & ... & w_1^{(k)}(T-1) \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ w_{Dim_X}^{(1)}(0) & ... & w_{Dim_X}^{(1)}(T-1) & ... & w_{Dim_X}^{(k)}(0) & ... & w_{Dim_X}^{(k)}(T-1) \end{bmatrix}$$

Where $w_{Dim}^{(i)}(j)$ the unknown process noise that is due to the conditions of the data gathering and $\hat{W}_-$ an element of the matrix Zonotope M_w (a concatenation of multiple noise Zonotopes Z_w):

$$M_w =< C_{M_w} = [c_{Z_w} ... c_{Z_w}], G_{M_w} = [G_{M_w}^{(1)} ... G_{M_w}^{(\gamma_{M_w})}] > \tag{6}$$

Furthermore, We denote R_N the exact reachable set [4] of a system after N time-steps as the set of all trajectories of the system under input $u(i) \in Z_u$ and noise $w(i) \in Z_w$, starting from the initial set X_0:

$$R_N = \{x(N) \in \mathbb{R}^n | \forall i \in \{0, ..., N-1\} : x(i+1) = f(x(i), u(i)) + w(i), x(0) \in X_0, u(i) \in Z_u, w(i) \in Z_w\} \tag{7}$$

Our analysis aims to compute a set R'_N that over-approximates the exact reachable set R_N using data $D = \{U_-, X\}$ gathered from simulations of the system, to be more specific.

4 Description of the Investigation

All the data-driven reachability analysis algorithms defined below are inspired from [4] and [12].

4.1 Linear Time Invariant systems

A linear time invariant (LTI) system is a system that is both linear and time-invariant. A linear system is one in which the output is a linear combination of the input and the system's impulse response. A time-invariant system is one whose output does not change when the input is shifted in time. We consider the following discrete-time linear system:

$$x(i+1) = Ax(i) + Bu(i) + w(i) \tag{8}$$

Where A and B the unknown coefficients we aim to over-approximate, we can find multiple matrices A and B that are consistent with D as a result of the process noise. we seek to compute a set M_Σ that contains all $[A\ B]$ that are consistent with the data D given the process noise, we would then propagate the system forward using M_Σ for a desired number of steps.

Consider k simulated trajectories from system 8 as our data $D = \{U_-, X\}$, the process noise $Z_w \in M_w$ where M_w matrix zonotope containing multiple W_- and the set M_Σ containing all $[A\ B]$ that are consistent with D:

$$\forall\ [A\ B] \in M_\Sigma\ \ \exists\ W_- \in M_w : X_+ = AX_- + BU_- + W_- \tag{9}$$

The exact reachable set R_N can be computed using the correct coefficients A and B:

$$R_{i+1} = [A\ B](R_i \times Z_u) + Z_w \qquad R_0 = X_0 \tag{10}$$

We solve for M_Σ consistent with data D given that $\begin{bmatrix} X_- \\ U_- \end{bmatrix}$ has a right-inverse:

$$X_+ = M_\Sigma \begin{bmatrix} X_- \\ U_- \end{bmatrix} + M_W \quad \Rightarrow \quad M_\Sigma = (X_+ - M_w) \begin{bmatrix} X_- \\ U_- \end{bmatrix}^{-1}$$

We introduce **Algorithm 2**:

Algorithm 2 Reachability of Linear Time-invariant system

Input: data from trajectories $D = \{U_-, X\}$, number of steps N, initial set zonotope X_0, noise Zonotope Z_w, input zonotope Z_u and matrix zonotope M_w.

Output: Reachable sets R'

$R' \leftarrow [X_0]$

$M_\Sigma \leftarrow (X_+ - M_w) \begin{bmatrix} X_- \\ U_- \end{bmatrix}^{-1}$

for $i \leftarrow 0$ to $N-1$ **do**

$R'_{i+1} \leftarrow M_\Sigma(R'_i \times Z_u) + Z_w$

end for

Since the correct set of coefficients $[A\ B] \in M_\Sigma$ and both R_N and R'_N start from the initial set X_0, then R'_N over-approximate the exact reachable set R_N.

4.2 Lipschitz Nonlinear systems

A Lipschitz nonlinear systems is a type of mathematical system that is used to describe the behavior of dynamical systems. This system is nonlinear in nature, meaning that the output of the system is not directly proportional to the input. Instead, the output is a function of the input, which can be represented by a set of differential equations. The Lipschitz condition is a mathematical condition that ensures that the system is well-behaved and will not produce chaotic behavior. We consider the following Lipschitz nonlinear system where we assume f to be twice differentiable:

$$x(i+1) = f(x(i), u(i)) + w(i) \tag{11}$$

A local linearization of the system is perform by a Taylor series expansion at the linearization point $z^* = \begin{bmatrix} x^* \\ u^* \end{bmatrix}$:

$$f(z) = f(z^*) + \frac{\partial f(z)}{\partial z}|_{z=z^*}(z - z^*) + \frac{1}{2}(z - z^*)^T \frac{\partial^2 f(z)}{\partial z^2}|_{z=z^*}(z - z^*) + ... \tag{12}$$

Which we can over-approximate by a first order Taylor series and a remainder $L(z)$:

$$f(z) \in f(z^*) + \frac{\partial f(z)}{\partial z}|_{z=z^*}(z - z^*) + L(z) \tag{13}$$

We then rewrite the system equation like so:

$$f(x,u) = f(x^*, u^*) + \underbrace{\frac{\partial f(x,u)}{\partial x}|_{x=x^*,u=u^*}}_{\overline{A}}(x - x^*) + \underbrace{\frac{\partial f(x,u)}{\partial u}|_{x=x^*,u=u^*}}_{\overline{B}}(u - u^*) + L(x,u)$$

$$f(x,u) = \begin{bmatrix} f(x^*,u^*) & \overline{A} & \overline{B} \end{bmatrix} \begin{bmatrix} 1 \\ x - x^* \\ u - u^* \end{bmatrix} + L(x,u) \tag{14}$$

We will use the least-squares approach to obtain a linearized model M'. $L(z)$ can be computed as $L(z) = \frac{1}{2}(z - z^*)^T \frac{\partial^2 f(z)}{\partial z^2}|_{z=z^*}(z - z^*)$, given that the model is known. Since the system model is unknown in our case, we instead make the common assumption that f is Lipschitz continuous to over-approximate $L(z)$ from data D:

$$f : F \to \mathbb{R}^n \text{ Lipschitz continuous } \Rightarrow \forall z, z' \in F,\, \exists L^* \geq 0 : ||f(z) - f(z')||_2 \leq L^* ||z - z'||_2 \tag{15}$$

We also denote the covering radius δ such as $\forall z \in F\ \exists z_i = \begin{bmatrix} (X_-)_{:,i} \\ (U_-)_{:,i} \end{bmatrix} \in D$ such that $||z - z_i|| \leq \delta$. Given $\Delta M' = \begin{bmatrix} f(z^*) & \overline{A} & \overline{B} \end{bmatrix}$ the model mismatch, we now need to over-approximate $f(z) - \Delta M'$, 14 becomes:

$$f(z) = M' \begin{bmatrix} 1 \\ z - z^* \end{bmatrix} + \underbrace{\Delta M' \begin{bmatrix} 1 \\ z - z^* \end{bmatrix} + L(z)}_{Z_L + Z_e \supseteq}$$

Then, we consider all $z_i \in D$, we have, given some value for $(W_-)_{:,i} \in Z_w$ where we omit i for simplicity.

$$(X_+)_{:,i} - (W_-)_{:,i} = (M' + \Delta M') \begin{bmatrix} 1 \\ z_i - z^* \end{bmatrix} + L(z_i)$$

$$\Delta M' \begin{bmatrix} 1 \\ z_i - z^* \end{bmatrix} + L(z_i) \in \underbrace{\overset{\Rightarrow}{(X_+)_{:,i} - z_w - M' \begin{bmatrix} 1 \\ z_i - z^* \end{bmatrix}}}_{Z_L \supseteq}$$

In this manner, the model mismatch and the nonlinearity term at one data point z_i can be over-approximated. We now need to find Z_L, which can be computed as per algorithm 3 by finding $\underline{l}$ and $\bar{l}$. Having our Lipschitz continuity assumption 15 and covering radius δ, we can find that:

$$\exists z_i \in D : ||f(z) - f(z_i)|| \leq L_* ||z - z_i|| \leq L^* \delta \tag{16}$$

We can now say:

$$f(z) \in M' \begin{bmatrix} 1 \\ z_i - z^* \end{bmatrix} + Z_L + Z_e \tag{17}$$

Where $Z_e =< 0, diag(L^*\delta, ..., L^*\delta) >$, note that computing an upper bound on L^* and δ from data D is a non-trivial task [4]:

$$L^* = max_{z_i, z_j \in D, i \neq j} \frac{||f(z_i) - f(z_j)||}{||z_i - z_j||} \qquad \delta = max_{z_i \in D} min_{z_j \in D, i \neq j} ||Z_i - Z_j||$$

Algorithm 3 Reachability of Lipschitz nonlinear system

Input: data from trajectories $D = \{U_-, X\}$, number of steps N, initial set zonotope X_0, noise Zonotope Z_w, input zonotope Z_u, matrix zonotope M_w, Lipschitz constant L^* and covering radius δ .

Output: Reachable sets R'

$R' \leftarrow [X_0]$

$M' \leftarrow (X_+ - C_{M_w}) \begin{bmatrix} 1_{1 \times T} \\ X_- - 1 \otimes x^* \\ U_- - 1 \otimes u^* \end{bmatrix}^{-1}$

$\bar{l} \leftarrow max_j((X_+)_{:,j} - M' \begin{bmatrix} 1 \\ (X_-)_{:,j} - x^* \\ (U_-)_{:,j} - u^* \end{bmatrix})$

$\underline{l} \leftarrow min_j((X_+)_{:,j} - M' \begin{bmatrix} 1 \\ (X_-)_{:,j} - x^* \\ (U_-)_{:,j} - u^* \end{bmatrix})$

$Z_L \leftarrow zonotope(\underline{l}, \bar{l}) - Z_w$

$Z_e \leftarrow < 0, diag(L^*\delta, ..., L^*\delta) >$

for $i \leftarrow 0$ to $N - 1$ **do**

$R'_{i+1} \leftarrow M'(1 \times R'_i \times Z_u) + Z_w + Z_L + Z_e$

end for

Following from 17, given that the exact reachable set R_N and the computed set R'_N both start from the initial set X_0, then R'_N over-approximates R_N.

4.3 Polynomial systems

A polynomial system is a set of algebraic equations in which the unknowns are the coefficients of the polynomials for each dimension of the system. We consider the following polynomial system of dimension dim_p:

$$x(i+1) = f_p(x(i), u(i)) + w(i) \tag{18}$$

Given vector α where α_i represents the highest degree of equation i of the polynomial system and C matrix containing all the coefficients of the monomials for each equation of the system:

$$f_p(x(i), u(i)) = \begin{bmatrix} \sum_{j=0}^{\alpha_1} C_{1,j} \begin{bmatrix} x(i) \\ u(i) \end{bmatrix}^j \\ \vdots \\ \sum_{j=0}^{\alpha_{dim_p}} C_{dim_p,j} \begin{bmatrix} x(i) \\ u(i) \end{bmatrix}^j \end{bmatrix} \tag{19}$$

We denote that given an upper bound on the degree of all equation in 19, we can determinate all the monomials of the system considering that some coefficient might be null. we define the monomials m(x(i), u(i)) of the system as a vector containing all combinations of x(i) and u(i) both of a power up to the upper bound. We can clearly notice that unknown coefficient attached to those monomials are the key to computing an over-approximation. To find those coefficients, we first need to rewrite f_p like so:

$$f_p(x(i), u(i)) = Cm(x(i), u(i)) \tag{20}$$

We aim to compute a set M_Σ^p that contains all C that are consistent with data D given the noise bound and propagate ahead using M_Σ^p starting from X_0. Let a matrix $\hat{G}$ with k input-state trajectories inserted as arguments to $m(.)$, as well as $I_{R_i'}^m$ the interval computations of all monomials of $m(x, u)$ while over-approximating R_i' by an interval. We get $\hat{G} = [m(x(0), u(0)) \ ... \ m(x(k-1), u(k-1))]$ and denote the matrix H^p such that $\hat{G}H^p = I$ given than $\hat{G}$ has a right-inverse. We can then write:

$$M_\Sigma^p = (X_+ - M_w)H^p \tag{21}$$

Algorithm 4 Reachability of Polynomial system

Input: data from trajectories $D = \{U_-, X\}$, data-base matrix $\hat{G}$, number of steps N, initial set zonotope X_0, noise Zonotope Z_w, input zonotope Z_u and matrix zonotope M_w.

Output: Reachable sets R'

$R' \leftarrow [X_0]$
$M_\Sigma^p \leftarrow (X_+ - M_w)\hat{G}^{-1}$
for $i \leftarrow 0$ to $N-1$ **do**
  $(\underline{m}_i) \leftarrow min_{x \in int(R_i'), u \in int(Z_u)} m(x, u)$
  $(\overline{m}_i) \leftarrow min_{x \in int(R_i'), u \in int(Z_u)} m(x, u)$
  $I_{R_i'}^m \leftarrow [\underline{m}, \overline{m}]$
  $R_{i+1}' \leftarrow M_\Sigma^p I_{R_i'}^m + Z_w$
end for

Similarly to (4.1), both the exact reachable set R_N and R_N' start from the initial set X_0 and $C \in M_\Sigma^p$, we can then say that R_N' over-approximates R_N.

5 Evaluation of the Investigation

To demonstrate the toolbox's accuracy at over-approximating the reachable set of a system, we introduce example systems for each type of analysed system:

1. Linear Time-Invariant System:

   We consider the data driven reachability of the Invariant time-continuous linear systems from [13]:

$$\dot{x}(i) = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} x(i) + \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} u(i) + w(i) \tag{22}$$

   More especially, we consider the discrete version of system (22), using a sampling step of 0.05 seconds:

$$x(i+1) = \begin{bmatrix} 1.0512 & 0 & 0 \\ 0.0525 & 1.0512 & 0 \\ 0.0013 & 0.0525 & 1.0512 \end{bmatrix} x(i) + \begin{bmatrix} 0.0512 \\ -0.0499 \\ -0.0012 \end{bmatrix} u(i) + w(i) \tag{23}$$

   We will use 1 trajectories of 100 steps as data D, with the process noise bounded to zonotope $Z_w =< \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0.005 & 0 & 0 \\ 0 & 0.005 & 0 \\ 0 & 0 & 0.005 \end{bmatrix} >$ and input bounded to zonotope $Z_u =< [10], [0.25] >$. We aim to compute 6 steps ahead of the initial set $X_0 =< \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0.15 & 0 & 0 \\ 0 & 0.15 & 0 \\ 0 & 0 & 0.15 \end{bmatrix} >$:

```
from Linear_sys import linear_sys
import numpy as np
from Zonotope import Zonotope
steps = 100
initpoints = 1
dim_x = 3
X0 = Zonotope(np.array(np.ones((dim_x, 1))),
    0.15 *np.diag(np.ones((dim_x, 1)).T[0]))
U = Zonotope(10, 0.25)
W = Zonotope(np.array(np.zeros((dim_x, 1))),
    0.005 * np.ones((dim_x, 1)))
A = np.array([[1, 0, 0], [1, 1, 0],[0, 1, 1]])
B_ss = np.array([1, -1, 0])
C = np.array([1, 0, 0])
D = 0
L_sys = linear_sys(A, B_ss, C, D, X0, U, W,
    dim_x, initpoints, steps, samplingtime=0.05)
model, X_data = L_sys.Run_reachability(6, plot=True)
```

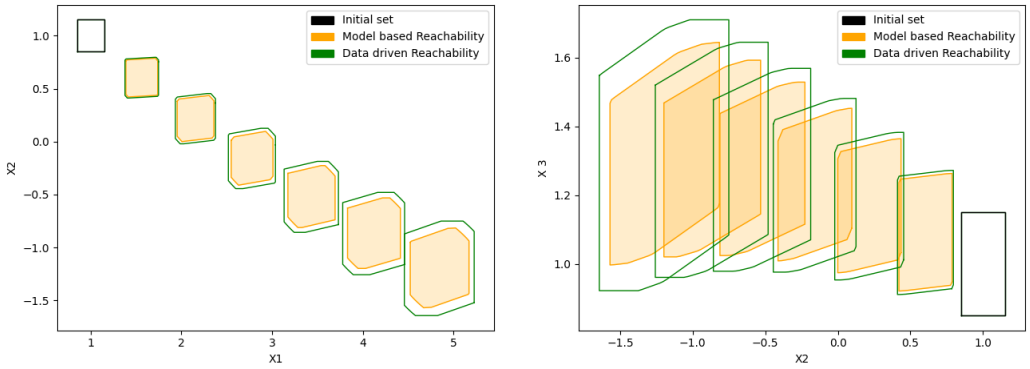


Figure 4: The model based reachability represents the exact reachable set R_N of the system (23) as it was computed using the right coefficients A and B, the data-driven reachable set R' correctly over-approximate the exact reachable set for each step of the analysis

2. Lipschitz Nonlinear systems:

To demonstrate the usability of data-driven reachability analysis of Lipschitz nonlinear systems, we consider the discrete version of a continuous stirred tank reactor (CSTR) [14], We gather 20 trajectories of 25 steps as the data D, with process noise bounded by zonotope $Z_w =< \begin{bmatrix}0\\0\end{bmatrix}, \begin{bmatrix}10^{-4} & 0\\0 & 10^{-4}\end{bmatrix} >$, input bounded to zonotope $Z_u =< \begin{bmatrix}0.01\\0.01\end{bmatrix}, \begin{bmatrix}0.1 & 0\\0 & 0.2\end{bmatrix} >$. We aim to compute 5 steps ahead of the initial set $R_0 =< \begin{bmatrix}-1.9\\-20\end{bmatrix}, \begin{bmatrix}0.005 & 0\\0 & 0.3\end{bmatrix} >$:

```
from NonLinear_sys import NonLinear_sys, cstrdiscr
import numpy as np
from Zonotope import Zonotope
dim_x = 2
U = Zonotope(np.array(np.array([0.01, 0.01])
    .reshape((2, 1))),np.diag([0.1, .2]))
R0 = Zonotope(np.array([-1.9, -20]).reshape(
    (dim_x, 1)),np.diag([0.005, .3]))
dt = 0.015
initpoints = 20
steps = 25
wfac = 10**-4
nl_sys = NonLinear_sys(dt, U, R0, wfac, dim_x,
    initpoints, steps, cstrdiscr)
data = nl_sys.Data_Driven_Reachability(5,
    ZepsFlag=True, plot=True)
```

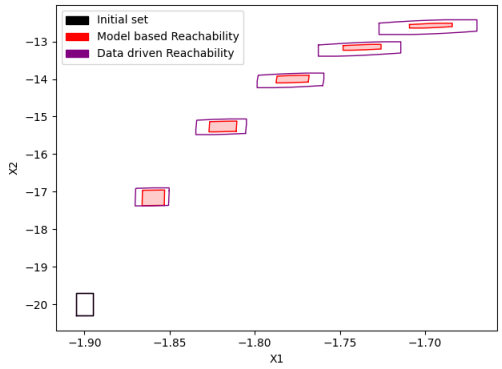


Figure 5: Similarly to the evaluation in subsection (5.1), the model-based reachability analysis represent the exact reachable set R_N of the system. Here again, the reachable sets computed using the data gathered D correctly over-approximate R_N.

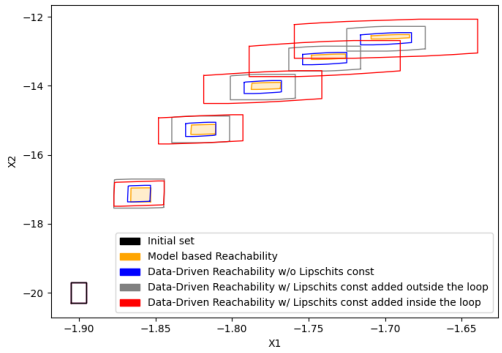


Figure 6: We introduce a variation of the data-driven reachability analysis performed in figure 5, where we add the Lipschitz constant L after all the propagation is done, instead of adding it at each step of the propagation. We visualize this variation, as well as the computed reachable sets without L. The set computed using the constant L at each step (represented in red), cumulatively adds L within the propagation, which results in a larger over-approximation.

3. Polynomial systems:

We consider the data-driven reachability of a time-invariant polynomial system from [4]:

$$f_p(x,u) = \begin{bmatrix} 0.7x_1 + u_1 + 0.32x_1^2 \\ 0.09x_1 + 0.32u_2x_1 + 0.4x_2^2 \end{bmatrix} \tag{24}$$

Which we use to compute 1 trajectory of 7 steps as our data D, with the process noise bounded by a zototope $Z_w =< \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 10^{-1} & 0 \\ 0 & 10^{-4} \end{bmatrix} >$, the input bounded by a zonotope $Z_u =< \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix}, \begin{bmatrix} 0.01 & 0 \\ 0 & 0.02 \end{bmatrix} >$ and the initial set $R_0 =< \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 0.05 & 0 \\ 0 & 0.3 \end{bmatrix} >$, We aim to compute the 3 first steps ahead of R_0:

```
from Poly_sys import Poly_sys, poly_func
import numpy as np
from Zonotope import Zonotope
N = 3
dt = 0.015
U = Zonotope(np.array(np.array([0.2, 0.3])
    .reshape((2, 1))), np.diag([0.01, .02]))
R0 = Zonotope(np.array([1, 2]).reshape((2, 1))
    ,np.diag([0.05, .3]))
dim_x = 2
initpoints = 1
steps = 7
wfac = 0.0001
poly_sys = Poly_sys(dt, U, R0, wfac, dim_x,
    initpoints, steps, poly_func)
data = poly_sys.Data_Driven_Reachability(N, False)
```

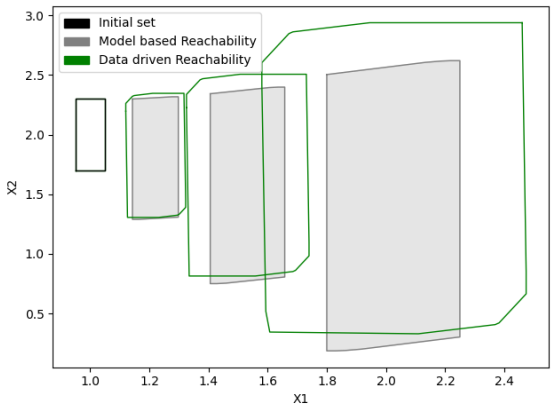


Figure 7: The model-based reachability represent the exact reachable set R_N of the system, we can notice that the data-driven computed reachable set R'_N doesn't completely enclose R_N, meaning that the reachable set R'_N not intersecting with an unsafe state gives us no guarantee about the system safety.

6 Conclusions

We aim to implement algorithms for computing the reachable set of an unknown system using data gather from simulations of that system, we first introduce the concept of reachability analysis, and expend it to the data-driven approach, we then discuss multiple approaches that tackle the same problem and we define notation and set representation needed for the analysis. Next, we describe the algorithms used to compute the reachable set of LTI systems (compute the set M_Σ containing all $[A\ B]$ that are consistent with the data D given the process noise), Lipschitz Nonlinear systems (compute a linear model of the system and over-approximate the model mismatch with the constant L) and Polynomial systems (Compute M_Σ^p that contains all the coefficients that are consistent

with the data D). we then illustrate the effectiveness of each algorithm by displaying and comparing the computed reachable set with the base model of the system.

References

[1] Amr Alanwar et al. “Enhancing Data-Driven Reachability Analysis using Temporal Logic Side Information”. In: *2022 International Conference on Robotics and Automation (ICRA)*. IEEE. 2022, pp. 6793–6799.

[2] SH Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry and Engineering. 1º Edição*. 2001.

[3] Anatole Katok and Boris Hasselblatt. *Introduction to the modern theory of dynamical systems*. 54. Cambridge university press, 1997.

[4] Amr Alanwar et al. “Data-Driven Reachability Analysis from Noisy Data”. In: *arXiv preprint arXiv:2105.07229* (2021).

[5] Matthias Althoff. “Reachability analysis and its application to the safety assessment of autonomous cars”. PhD thesis. Technische Universität München, 2010.

[6] Leonidas J Guibas, An Thanh Nguyen, and Li Zhang. “Zonotopes as bounding volumes.” In: *SODA*. Vol. 3. 2003, pp. 803–812.

[7] David K Arrowsmith, Colin M Place, CH Place, et al. *An introduction to dynamical systems*. Cambridge university press, 1990.

[8] Matthias Althoff, Goran Frehse, and Antoine Girard. “Set propagation techniques for reachability analysis”. In: *Annual Review of Control, Robotics, and Autonomous Systems* 4.1 (2021).

[9] Matthias Rungger and Paulo Tabuada. “Computing Robust Controlled Invariant Sets of Linear Systems”. In: *IEEE Transactions on Automatic Control* 62.7 (2017), pp. 3665–3670. DOI: 10.1109/TAC.2017.2672859.

[10] Xuejiao Yang and Joseph K Scott. “A comparison of zonotope order reduction techniques”. In: *Automatica* 95 (2018), pp. 378–384.

[11] Antoine Girard. “Reachability of uncertain linear systems using zonotopes”. In: *International Workshop on Hybrid Systems: Computation and Control*. Springer. 2005, pp. 291–305.

[12] Amr Alanwar et al. “Data-driven reachability analysis using matrix zonotopes”. In: *Learning for Dynamics and Control*. PMLR. 2021, pp. 163–175.

[13] Zanasi Roberto. *Reachability and Controllability*. 2016.

[14] D Limon et al. “Robust MPC of constrained nonlinear systems based on interval arithmetic”. In: *IEE Proceedings-Control Theory and Applications* 152.3 (2005), pp. 325–332.